

Proyecto 360

Informe de Despliegue en la nube

Integrantes

Javier Muñoz

Benjamin Contreras

Maximiliano Rubilar

1. Introducción:

El proyecto abarca la implementación de un frontend en React con autenticación mediante Microsoft Entra ID, un backend compuesto por microservicios en Spring Boot, y su exposición controlada a través de Amazon API Gateway, todo desplegado sobre una instancia EC2 de AWS.

A lo largo de este documento se detallan los pasos seguidos para configurar el proveedor de identidad (IDaaS), el API Gateway como capa de seguridad y enrutamiento, la infraestructura de cómputo en AWS, y la evidencia de funcionamiento del sistema completo, incluyendo la validación de tokens JWT en cada una de las rutas expuestas.

2. Objetivo del sistema pedidos 360

El objetivo del sistema es un panel de administración para el cliente, podrá gestionar usuarios, crear o editar productos, este proyecto fue desarrollado en React más Vite por el lado del frontend y por el backend se crearon cuatro microservicios con el framework Spring Boot cada uno con su propia base de datos que lograra separar tanto la lógica de negocio como los datos. Se creó un Tenant de microsoft para usar Microsoft Entra ID como servicio Identity as a service (IDaaS), con el objetivo de proteger el acceso a los microservicios y asegurar que solo usuarios autenticados puedan consumir sus endpoints a través del API Gateway de AWS conocido como API manager.

2.1 Arquitectura general

La arquitectura del sistema se compone de las siguientes capas:

- Frontend: aplicación React + Vite, que utiliza MSAL (Microsoft Authentication Library) para el login contra Azure AD y la obtención del token JWT.
- Identity as a Service (IDaaS): Microsoft Entra ID, donde se registró la aplicación, se definieron los scopes de la API y se administran los usuarios autorizados.
- API Gateway (AWS): actúa como punto de entrada único, valida el JWT mediante un authorizer, aplica CORS y enruta las peticiones hacia cada microservicio.

- Backend: tres microservicios en Spring Boot (users, items, categories), cada uno con su propia base de datos MySQL, desplegados en una instancia EC2.
- Infraestructura: una instancia EC2 de AWS aloja tanto el frontend (servido como build estático) como los tres microservicios y la base de datos MySQL.

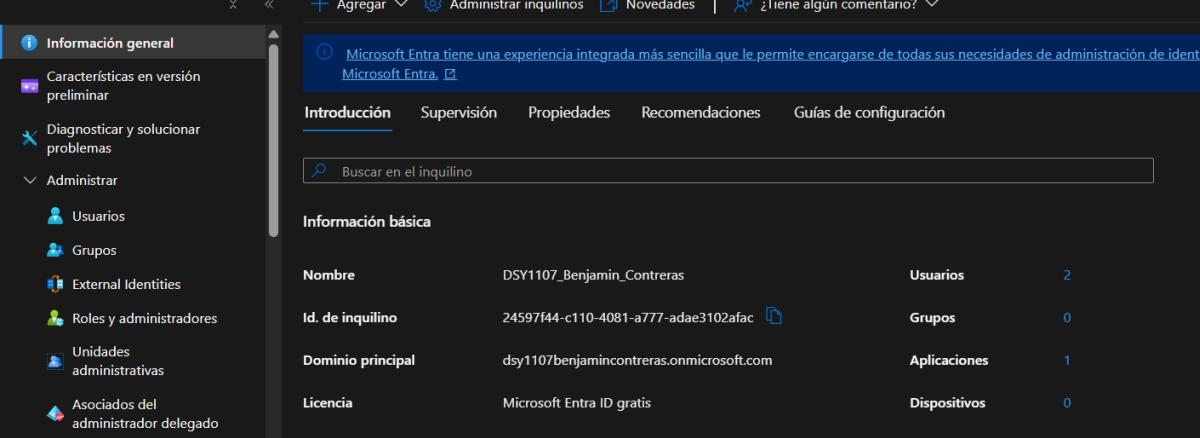
El flujo completo es: el usuario inicia sesión desde el frontend contra Microsoft Entra ID, obtiene un JWT, y con ese token consume los endpoints expuestos a través del API Gateway de AWS, el cual valida el token antes de reenviar la petición al microservicio correspondiente en la EC2.

3. Configuración del IDaaS:

Se utilizó Microsoft Entra ID como proveedor de identidad. A continuación se detalla la configuración del Tenant, los usuarios registrados, la aplicación creada y los scopes definidos para proteger la API.

3.1 Tenant

Se creó un Tenant propio de la organización (DSY1107_Benjamin_Contreras), que actúa como directorio privado para administrar la identidad de los usuarios del sistema. Este Tenant es el emisor (issuer) de los tokens JWT que luego se validan en el API Gateway.

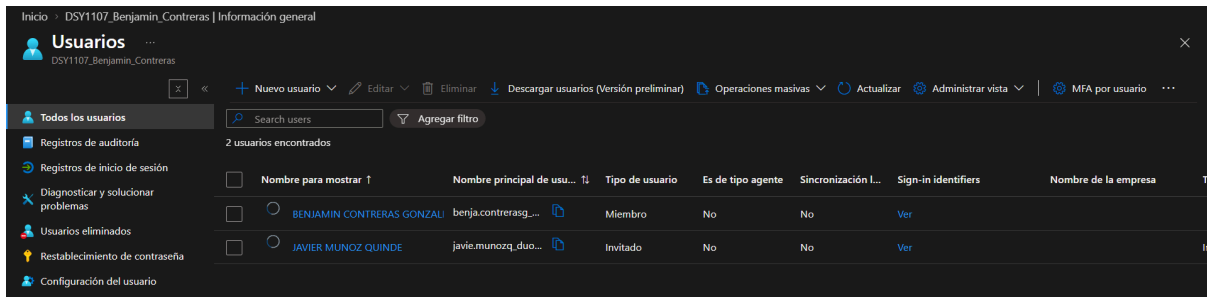


The screenshot shows the Microsoft Entra ID portal interface. On the left is a navigation pane with options like 'Información general', 'Características en versión preliminar', 'Diagnosticar y solucionar problemas', and 'Administrar'. The 'Administrar' section is expanded, showing 'Usuarios', 'Grupos', 'External Identities', 'Roles y administradores', 'Unidades administrativas', and 'Asociados del administrador delegado'. The main content area displays the 'Información básica' (Basic Information) for the tenant 'DSY1107_Benjamin_Contreras'. It includes a search bar and a table with the following data:

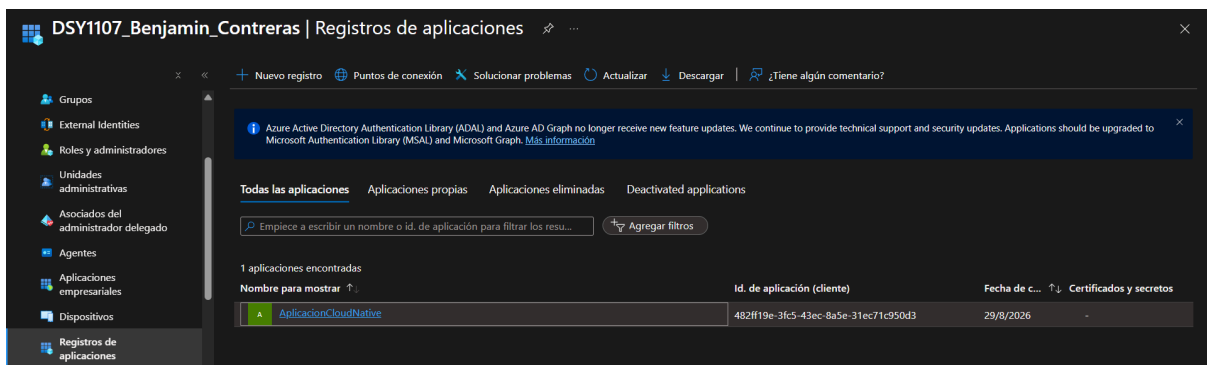
Información básica			
Nombre	DSY1107_Benjamin_Contreras	Usuarios	2
Id. de inquilino	24597f44-c110-4081-a777-adae3102afac	Grupos	0
Dominio principal	dsy1107benjamincontreras.onmicrosoft.com	Aplicaciones	1
Licencia	Microsoft Entra ID gratis	Dispositivos	0

3.2 Usuarios

Dentro del Tenant se registraron los usuarios que tendrán acceso al sistema. Cada usuario podrá iniciar sesión desde el frontend y obtener un token JWT válido para consumir la API.



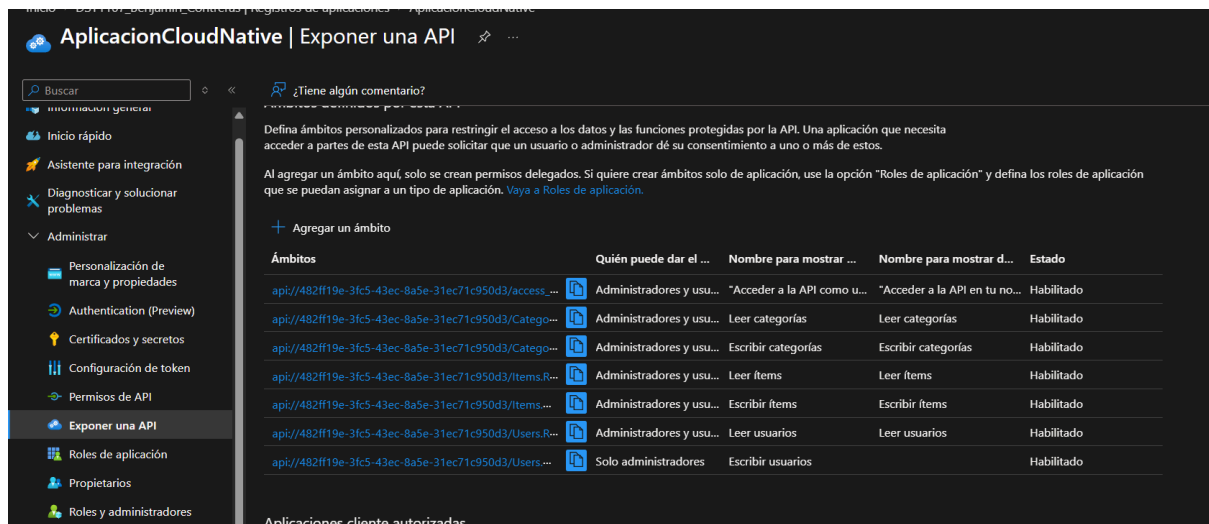
3.3 Creación de la aplicación:



Se registró la aplicación AplicacionCloudNative en el Tenant, la cual representa tanto al frontend (como cliente SPA) como a la API (como recurso protegido). Este registro entrega el Application (client) ID que utiliza MSAL para iniciar el flujo de autenticación.

3.4 scopes:

Se definieron scopes específicos para restringir el acceso a las distintas operaciones de la API, siguiendo el principio de menor privilegio. Cada scope habilita una operación de lectura o escritura sobre un recurso en particular, y el token JWT solo tendrá los scopes que el usuario haya solicitado y le hayan sido autorizados.



4. Flujo de autenticación OIDC con PKCE

El frontend utiliza el flujo Authorization Code con PKCE (Proof Key for Code Exchange) de OpenID Connect, gestionado automáticamente por la librería MSAL. Este flujo es el recomendado para aplicaciones públicas como una SPA, ya que no requiere almacenar un client secret en el navegador. Los pasos del flujo son:

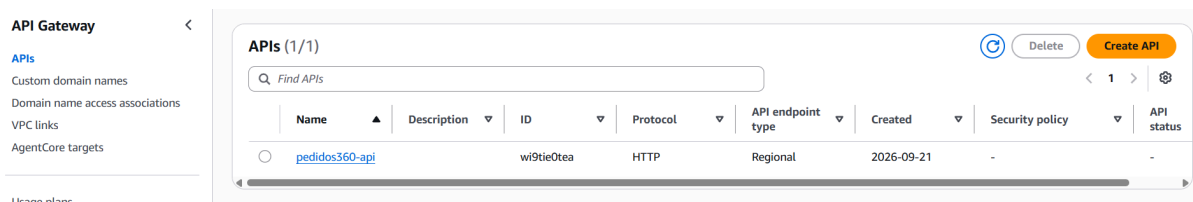
- 1. El usuario presiona "Iniciar sesión" y MSAL genera un code_verifier (secreto aleatorio) y su code_challenge (hash SHA-256 del verifier), además de los valores state y nonce.
- 2. El navegador es redirigido al endpoint de autorización de Microsoft Entra ID, enviando el code_challenge, el state y el nonce, junto con los scopes solicitados.
- 3. El usuario se autentica con sus credenciales en el formulario de Microsoft.
- 4. Entra ID redirige de vuelta al frontend con un authorization code de un solo uso.
- 5. MSAL valida que el state recibido coincida con el enviado, y luego canjea el code por los tokens, enviando también el code_verifier original.

- 6. Entra ID recalcula el hash del code_verifier y lo compara con el code_challenge recibido en el paso 2. Si coincide, entrega el id_token y el access_token.
- 7. MSAL valida la firma, el issuer, el audience y el nonce del id_token.
- 8. El frontend adjunta el access_token en el header Authorization: Bearer de cada llamada a la API.

Este mecanismo protege el intercambio de tokens frente a la interceptación del código de autorización, ya que un atacante que capture el code no podría canjearlo sin conocer el code_verifier original, que nunca se transmite por la red hasta el paso final.

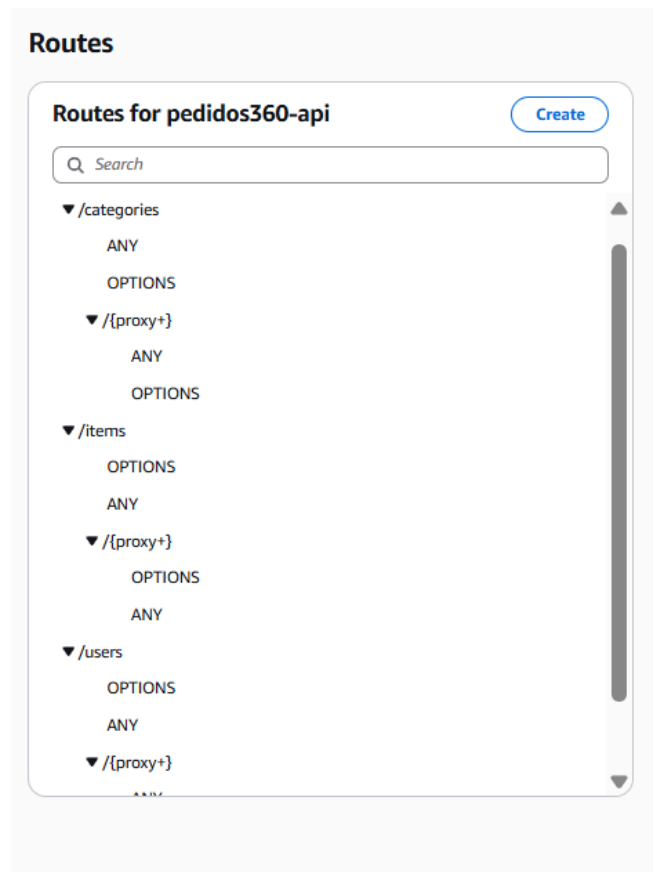
5. Creación de la API Gateway de AWS

El API manager o Api gateway consiste en un servicio completamente gestionado que actúa como el punto de entrada central para que las aplicaciones accedan a servicios backend, también tiene una capa de seguridad lo que es más importante para proteger los endpoints de los microservicios. Para que funcione la api gateway se tiene que crear las rutas específicas el método a utilizar y hacer una implementación que apunte a los endpoints (URI) de los microservicios. Luego se tiene que configurar los CORS para el tema de la seguridad.



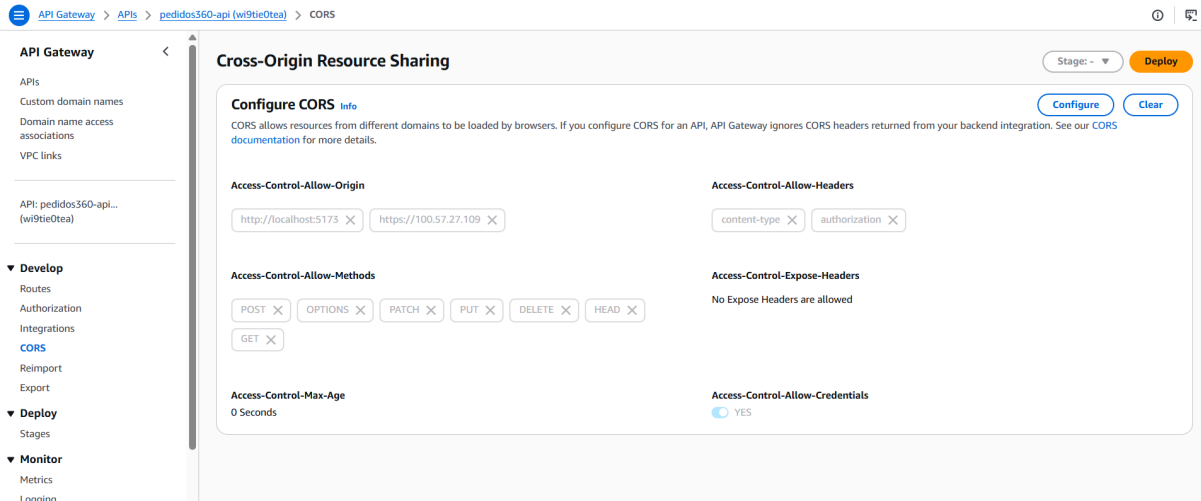
5.1 Rutas

Se configuraron rutas para cada uno de los tres recursos del backend (categories, items, users), cada una con su respectiva integración hacia el microservicio correspondiente desplegado en la EC2.



5.2 CORS:

Se configuró Cross-Origin Resource Sharing (CORS) a nivel de API para permitir que el frontend, servido desde la EC2, pueda realizar peticiones al API Gateway sin ser bloqueado por el navegador. Se autorizaron los orígenes del entorno de desarrollo y de producción, los métodos HTTP necesarios y el header Authorization, indispensable para el envío del token JWT.



5.3 JWT Authorizer (issues, audience):

Se configuró un authorizer de tipo JWT en el API Gateway, encargado de validar el token recibido en el header Authorization antes de reenviar la petición al backend. El authorizer valida que el token provenga del issuer correcto (el Tenant de Entra ID) y que el audience corresponda al Application ID de la API. Si el token no es válido, no está firmado correctamente, o el issuer/audience no coinciden, el API Gateway responde 401 sin siquiera llegar al microservicio.

The screenshot shows the 'Edit authorizer' configuration page in the Azure API Gateway console. The breadcrumb trail is 'API Gateway > APIs > pedidos360-api (w9t0e0tea) > Authorization'. The left sidebar shows navigation options: 'Custom domain names', 'Domain name access associations', 'VPC links', 'API: pedidos360-api... (w9t0e0tea)', 'Develop' (with sub-items: Routes, Authorization, Integrations, CORS, Reimport, Export), 'Deploy' (with sub-item: Stages), 'Monitor' (with sub-items: Metrics, Logging), and 'Protect' (with sub-item: Throttling). The main content area is titled 'Edit authorizer' and contains the following sections:

- Authorizer type**: Shows 'Authorizer ID' as 'smo0iz' and 'Authorizer type' as 'JWT'.
- Authorizer settings**:
 - Name**: 'Enter a name for the authorizer.' The input field contains 'azure-ad-authorizer'.
 - Identity source**: 'Enter a selection expression that defines the source of the token.' The input field contains '\$request.header.Authorization'.
 - Issuer URL**: 'Enter the issuer URL of your identity provider, commonly found in the issuer field of the Authorization Server's well-known metadata endpoint.' The input field contains 'https://sts.windows.net/24597f44-c110-4081-a777-adae3102afac/'.
 - Audience**: 'Enter client IDs that are registered with identity providers or any arbitrary string in the JWT audience claim that the authorizer must verify.' The input field contains 'api://482ff19e-3fc5-43ec-8a5e-31ec71c950d3'. There are 'Add audience' and 'Remove' buttons.

6. Validación JWT

Para cada ruta expuesta se validó el comportamiento del authorizer ante distintos escenarios: sin token, con token inválido y con token válido. La siguiente tabla resume los resultados esperados y verificados durante las pruebas del sistema:

Ruta	Sin token	Token inválido	Token válido
/users	401 Unauthorized	401 Unauthorized	200 OK
/items	401 Unauthorized	401 Unauthorized	200 OK
/categories	401 Unauthorized	401 Unauthorized	200 OK

A continuación se muestra evidencia capturada desde las herramientas de desarrollo del navegador de una petición exitosa hacia la ruta /users a través del API Gateway.

URL De Solicitud	https://wi9tie0tea.execute-api.us-east-1.amazonaws.com/users
Método De Solicitud	GET
Código De Estado	200 OK
Dirección Remota	35.169.38.40:443
Política De Referente	strict-origin-when-cross-origin
Encabezados de respuesta	
Access-Control-Allow-Credentials	true
Access-Control-Allow-Origin	https://100.57.27.109
ApiGW-Requestid	EELxtjrPoAMEblA=
Cache-Control	no-cache, no-store, max-age=0, must-revalidate
Content-Length	99
Content-Type	application/json
Date	Mon, 21 Sep 2026 22:57:53 GMT
Expires	0
Pragma	no-cache
Vary	Origin
Vary	Access-Control-Request-Method
Vary	Access-Control-Request-Headers
X-Content-Type-Options	nosniff
X-Frame-Options	DENY
X-Xss-Protection	0

[illegible]

Al remover el token o enviar uno inválido, el authorizer del API Gateway responde con código 401 Unauthorized antes de que la petición llegue al microservicio, lo que confirma que la validación del JWT ocurre efectivamente en la capa del API Gateway.

7. Creación de la EC2 de AWS con el frontend y backend

Para cumplir con el proyecto se creó una EC2 que tendrá desplegado tanto el frontend como el backend, nota importante para cumplir con el despliegue, es la que la EC2 tenga buena cantidad de ram, 1Gb de ram no será suficiente para poder ejecutar el proyecto de 4 microservicios en ejecución a la vez la ram estara al maximo y posiblemente el EC2 quede congelada, se requiere por lo minimo 4Gb de ram base.

Lanzar una instancia [Información](#)

Amazon EC2 le permite crear máquinas virtuales, o instancias, que se ejecutan en la nube de AWS. Comience rápidamente siguiendo los sencillos pasos que se indican a continuación.

Nombre y etiquetas [Información](#)

Nombre

[Agregar etiquetas adicionales](#)

Nombre asignado a la instancia: pedidos360-backend.

EC2 > Instancias > Lanzar una instancia

Utilice el campo de búsqueda o elija **Buscar más AMI**.

🔍 Busque en nuestro catálogo completo que incluye miles de imágenes de sistemas operativos y aplicaciones

Inicio rápido

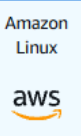
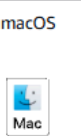
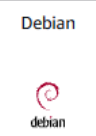
 Amazon Linux	 macOS	 Ubuntu	 Windows	 Red Hat	 SUSE Linux	 Debian	 Buscar más AMI Incluidas las AMI de AWS, Marketplace y la comunidad
---	--	---	--	--	---	--	---

Imagen de máquina de Amazon (AMI)

AMI de Amazon Linux 2023 kernel-6.18
ami-0fef201115eefe936 (64 bits (x86), uefi-preferred) / ami-0eb45f74aa8a20238 (64 bits (Arm), uefi)
Virtualización: hvm Habilitado para ENA: true Tipo de dispositivo raíz: ebs

Apto para la capa gratuita ▼

Descripción

Amazon Linux 2023 (kernel-6.18) es un sistema operativo moderno y de uso general basado en Linux que contará con soporte hasta junio de 2029. Está optimizado para AWS y diseñado para proporcionar un entorno de ejecución seguro, estable y de alto desempeño para desarrollar y ejecutar sus aplicaciones en la nube.

Amazon Linux 2023 AMI 2023.12.20260918.0 x86_64 HVM kernel-6.18

Selección de la imagen de máquina (AMI): Amazon Linux 2023.

▼ Tipo de instancia

Información | Obtener asesoramiento

Tipo de instancia

t2.large

Familia: t2 2 vCPU 8 GiB Memoria Generación actual: true

Bajo demanda Windows base precios: 0.1208 USD por hora

Bajo demanda Ubuntu Pro base precios: 0.0963 USD por hora

Bajo demanda RHEL base precios: 0.1216 USD por hora

Bajo demanda SUSE base precios: 0.1928 USD por hora Bajo demanda Linux base precios: 0.0928 USD por hora

Todas las generaciones

Comparar tipos de instancias

Se aplican costos adicionales a las AMI con software preinstalado

Tipo de instancia seleccionado: *t2.large* (2 vCPU, 8 GiB de memoria), que cumple con el requisito mínimo de RAM.

▼ Configuraciones de red

Información

Red

Información

vpc-0bcc72123e1a0e998

Subred

Información

No hay preferencia (subred predeterminada en cualquier zona de disponibilidad)

Asignación automática de IP pública

Información

Habilitar

Se aplican cargos adicionales cuando no se cumplen los límites del nivel gratuito

Firewall (grupos de seguridad)

Información

Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para p instancia.

● Crear un grupo de seguridad

○ Seleccionar un grupo de seguridad existente

Crearemos un nuevo grupo de seguridad denominado "launch-wizard-1" con las siguientes reglas:

✓ Permitir el tráfico de SSH desde

Ayuda a establecer conexión con la instancia

Cualquier lugar

0.0.0.0/0

✓ Permitir el tráfico de HTTPS desde Internet

Para configurar un punto de enlace, por ejemplo, al crear un servidor web

✓ Permitir el tráfico de HTTP desde Internet

Para configurar un punto de enlace, por ejemplo, al crear un servidor web

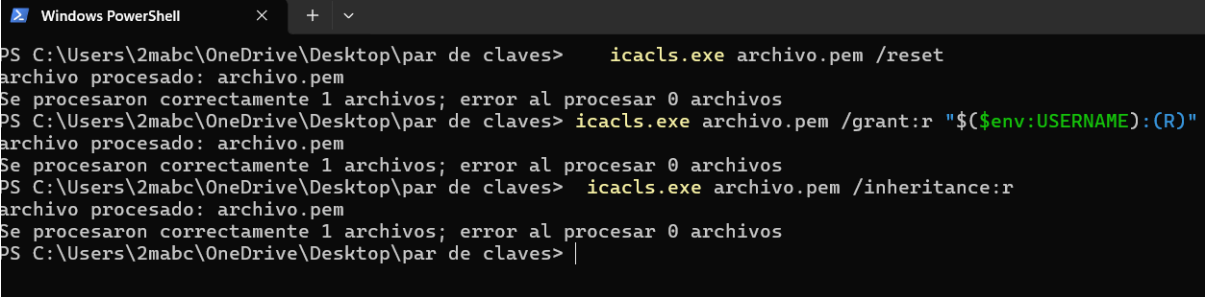
Configuración de red: IP pública habilitada y grupo de seguridad con tráfico de entrada permitido para SSH, HTTPS y HTTP.

8. Conectarse a la EC2 por SSH

Una vez lanzada la instancia, se accede a ella mediante SSH utilizando el par de llaves (.pem) generado al momento de crearla. Los pasos seguidos fueron los siguientes:

8.1 Ajustar permisos de la llave

Desde tu terminal, entra a la carpeta donde guardaste el .pem y ajustar los permisos (solo la primera vez, obligatorio o SSH lo rechaza):

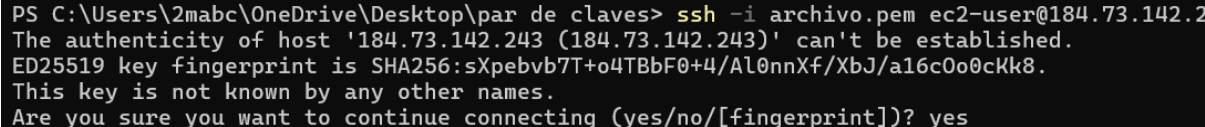


```
PS C:\Users\2mabc\OneDrive\Desktop\par de claves> icacls.exe archivo.pem /reset
archivo procesado: archivo.pem
Se procesaron correctamente 1 archivos; error al procesar 0 archivos
PS C:\Users\2mabc\OneDrive\Desktop\par de claves> icacls.exe archivo.pem /grant:r "$($env:USERNAME):(R)"
archivo procesado: archivo.pem
Se procesaron correctamente 1 archivos; error al procesar 0 archivos
PS C:\Users\2mabc\OneDrive\Desktop\par de claves> icacls.exe archivo.pem /inheritance:r
archivo procesado: archivo.pem
Se procesaron correctamente 1 archivos; error al procesar 0 archivos
PS C:\Users\2mabc\OneDrive\Desktop\par de claves> |
```

Ajuste de permisos del archivo .pem con icacls.exe en PowerShell, restringiendo el acceso solo al usuario actual.

8.2 Conexión

Conéctate usando la IP pública de tu instancia (la ves en la consola de EC2, columna "Public IPv4 address"):



```
PS C:\Users\2mabc\OneDrive\Desktop\par de claves> ssh -i archivo.pem ec2-user@184.73.142.2
The authenticity of host '184.73.142.243 (184.73.142.243)' can't be established.
ED25519 key fingerprint is SHA256:sXpevbv7T+o4TBbF0+4/Al0nnXf/XbJ/a16c0o0cKk8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

Conexión SSH inicial: aceptación de la huella del host.

8.3 Instalación de Java

Una vez dentro de la instancia, se instala el entorno de ejecución Java 17, necesario para correr los microservicios de Spring Boot (compilados como .jar ejecutables).

[illegible]

Instalación de Java 17 (Amazon Corretto) mediante el gestor de paquetes yum.

8.4 Verificación

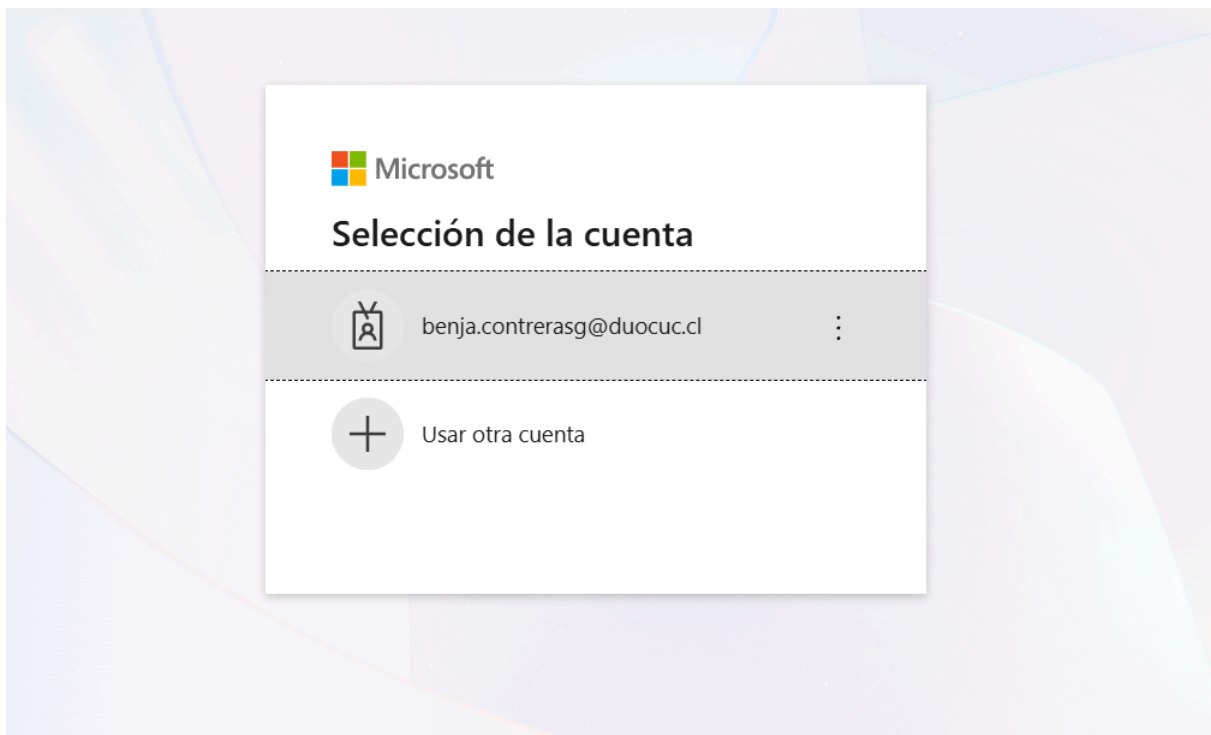
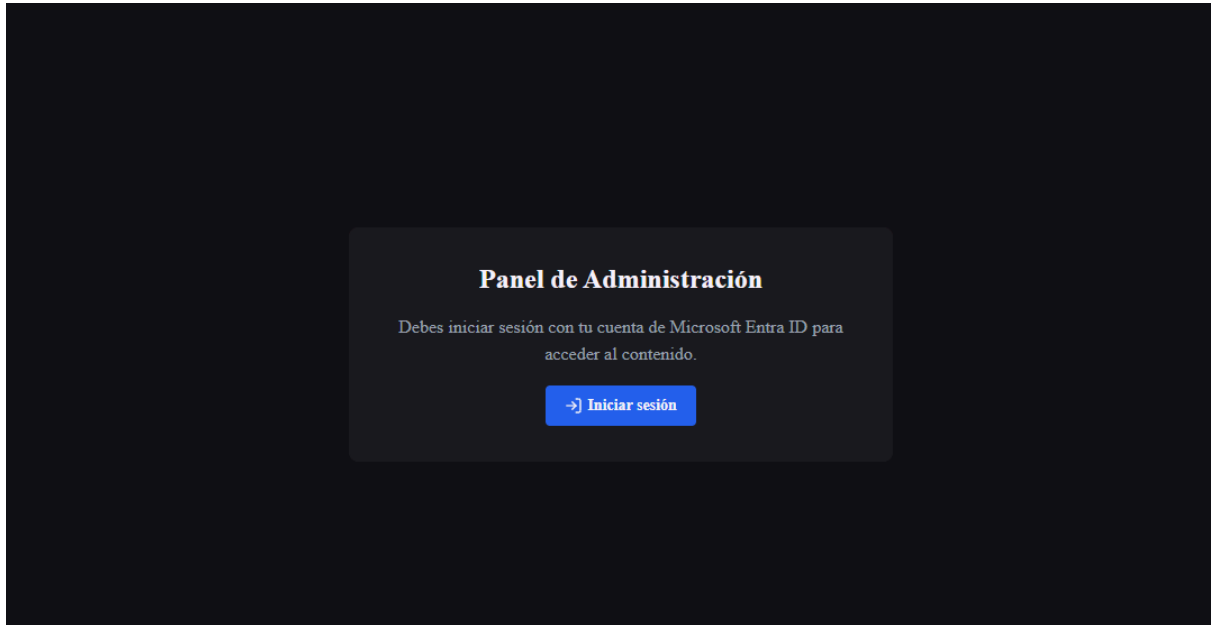
Finalmente, se verifica que la instalación se haya completado correctamente consultando la versión de Java instalada.

```
Complete!
[ec2-user@ip-172-31-29-47 ~]$ java -version
openjdk version "17.0.20.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-17.0.20.10.1 (build 17.0.20.1+10-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.20.10.1 (build 17.0.20.1+10-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-29-47 ~]$ |
```

Verificación de la instalación: OpenJDK 17.0.20.1 (Amazon Corretto).

9. Sistema de pedidos 360 funcionando

A continuación se presenta evidencia del sistema completo funcionando de manera integrada: autenticación, panel de administración y edición de datos, consumidos a través del API Gateway.



Pantalla inicial del Panel de Administración: solicita iniciar sesión con la cuenta de Microsoft Entra ID antes de permitir el acceso al contenido.

Panel de Administración

Usuarios, categorías e ítems, conectados vía el API Gateway en el puerto 8089.

[Usuarios](#)[Ítems](#)[Categorías](#)

Usuarios

1 registrados

 [+ Nuevo usuario](#)

Nombre	Correo	Estado	Creado	
benja	benja@gmail.com	• Activo	20-09-2026, 02:59 a. m.	 

Panel de Administración autenticado, mostrando la lista de usuarios registrados y confirmando que el frontend consume los datos vía el API Gateway.



Editar ítem

Nombre

Cantidad

1

Precio

2000

Cancelar

[Guardar cambios](#)

Formulario de edición de un ítem, parte de la gestión de productos del sistema.

10. Conclusiones

El desarrollo de este proyecto permitió implementar de forma práctica una arquitectura cloud native completa, integrando un proveedor de identidad externo (Microsoft Entra ID) con un backend de microservicios protegido por un API Gateway gestionado (AWS API Gateway), todo desplegado sobre infraestructura como servicio (EC2).

Entre las principales dificultades enfrentadas durante el despliegue se encuentran: la necesidad de contar con suficiente memoria RAM en la instancia EC2 para ejecutar simultáneamente los cuatro microservicios sin que el sistema se congelara, la correcta configuración de CORS para evitar bloqueos del navegador en las peticiones hacia el API Gateway, y la persistencia de los procesos backend ante reinicios de la instancia, resuelta finalmente mediante la configuración de servicios systemd.

El resultado final es un sistema funcional donde el acceso a cada endpoint del backend está protegido mediante validación de JWT en la capa del API Gateway, cumpliendo con los objetivos de seguridad y separación de responsabilidades planteados en la arquitectura cloud native del proyecto.